Universidad de Alcalá

Transacciones HTTP habituales

Esquemas de peticiones, respuestas y cabeceras

Arquitectura de Redes · Arquitectura de Redes I

La capa de aplicación: la Web y HTTP

Enrique de la Hoz de la Hoz
Universidad de Alcalá · Curso 2026-27

Versión 1.1 · 19-oct-2026

Convenciones

Cada transacción se representa como un diagrama de secuencia: el tiempo avanza hacia abajo, las peticiones (trazo continuo) van del cliente al servidor y las respuestas (trazo discontinuo) del servidor al cliente; la inclinación de las flechas recuerda que todo mensaje tarda en llegar. El contenido de cada mensaje, numerado como su flecha, aparece a la derecha. Los mensajes están simplificados: se conservan la línea de inicio y las cabeceras relevantes, se omiten las secundarias (Date, Server, Accept-Encoding…) y el cuerpo se resume entre ángulos. Las cabeceras que explican cada transacción aparecen **en negrita** sobre fondo gris; ⟨*CR LF*⟩ indica la línea en blanco que separa cabeceras y cuerpo. Las líneas horizontales discontinuas marcan la apertura o el cierre de conexiones TCP. En los casos 3, 5, 7, 8, 9 y 10, una segunda figura muestra lo que pasa en el navegador (lo que ve el usuario, la caché o las cookies guardadas), con los mismos datos del esquema.

Los valores de las secciones 3, 4 y 6 proceden de peticiones reales (curl, 15 de octubre de 2026) y los de la sección 10, de la captura de referencia de la práctica GP23; los demás son ejemplos coherentes con ellos. Al final de cada sección se indica, cuando existe, la actividad correspondiente de la web de prácticas (capturas para leer en Wireshark y secuencias para dibujar, con corrección automática) y la visualización interactiva del catálogo de la asignatura.

Índice

1 GET de una página: 200 OK . . . 2
2 Página con una imagen, conexión persistente . . . 3
3 Redirección a HTTPS: 301 Moved Permanently . . . 4
4 Objeto inexistente: 404 Not Found . . . 5
5 Envío de un formulario: POST . . . 6
6 Solo las cabeceras: HEAD . . . 7
7 GET condicional: 304 Not Modified . . . 8
8 Cookies: estado sobre un protocolo sin estado . . . 9
9 Autorización básica: 401 Unauthorized . . . 10
10 Inicio de sesión por formulario: POST + cookie de sesión . . . 11

Esta obra está bajo una licencia Creative Commons "Atribución-CompartirIgual 4.0 Internacional".

El logotipo de la Universidad de Alcalá es propiedad de la Universidad y no está sujeto a esta licencia.

1. GET DE UNA PÁGINA: 200 OK

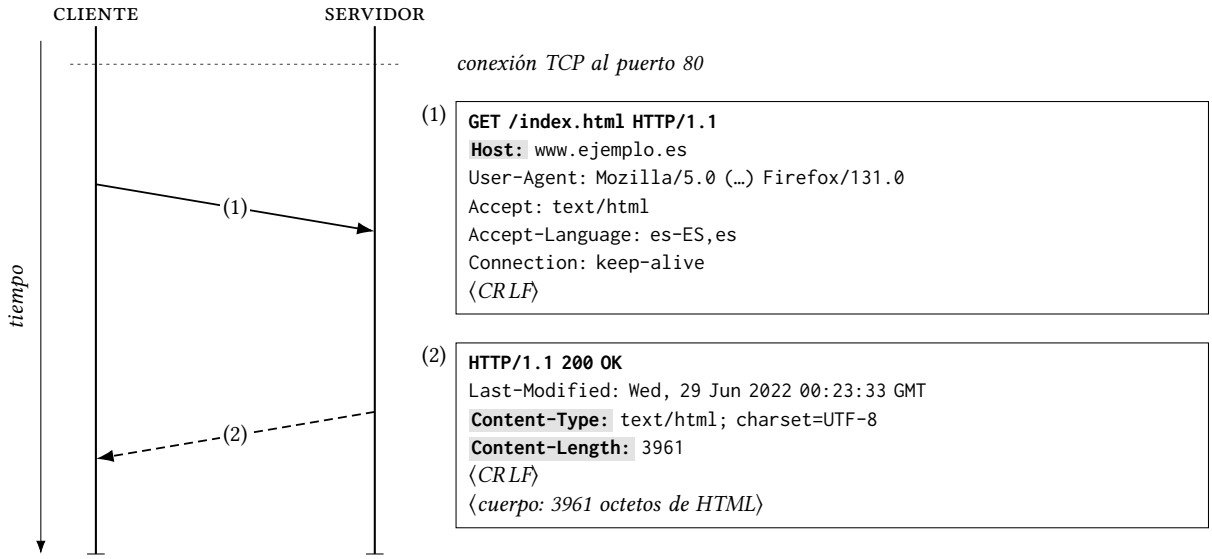


FIGURA 1. GET de una página: 200 OK.

OBSERVACIONES

(a) Host es obligatoria en HTTP/1.1: en una IP puede haber muchas webs.

(b) Content-Type dice qué llega (el navegador decide si lo muestra o lo descarga).

(c) Content-Length dice dónde acaba el cuerpo.

WEB DE PRÁCTICAS (`ar1.threatinteluah.com`):
captura: «Leer una petición y respuesta HTTP» (`CAP_HTTP_BASICA`)
secuencia: «Dibujar una visita web completa (DNS + HTTP)» (`SEC_HTTP_URL`)

2. Página con una imagen, conexión persistente

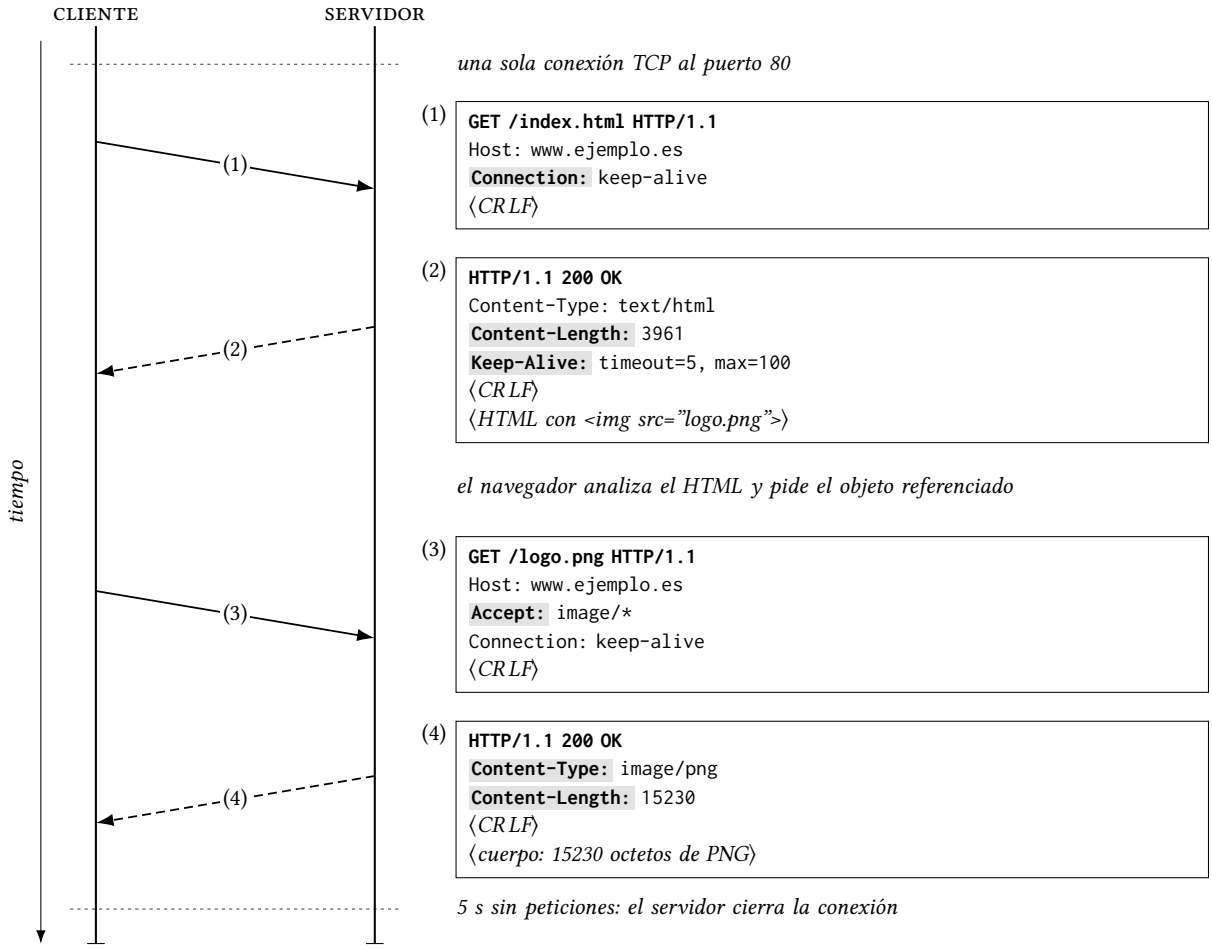


Figura 2. Página con una imagen, conexión persistente.

OBSERVACIONES

(a) Una petición y una respuesta por objeto, todas por la misma conexión TCP.

(b) Content-Length separa una respuesta de la siguiente.

(c) Keep-Alive: timeout=5 → el tiempo configurable tras el que se cierra.

WEB DE PRÁCTICAS (`ar1.threatinteluah.com`):
captura: «Leer el modo de las conexiones HTTP» (`CAP_HTTP_MODO`)
secuencia: «Dibujar una descarga HTTP, sin y con canalización» (`SEC_HTTP_BASICA`)
secuencia: «Dibujar y calcular el tiempo de descarga (RTT)» (`SEC_HTTP_TIEMPOS`)

VISUALIZACIÓN INTERACTIVA: *HTTP persistente: sin y con canalización*
`https://edelahozuah.github.io/computer-networks-visualizations/02-application-layer/http/http-persistent.es.html`

3. Redirección a HTTPS: 301 Moved Permanently

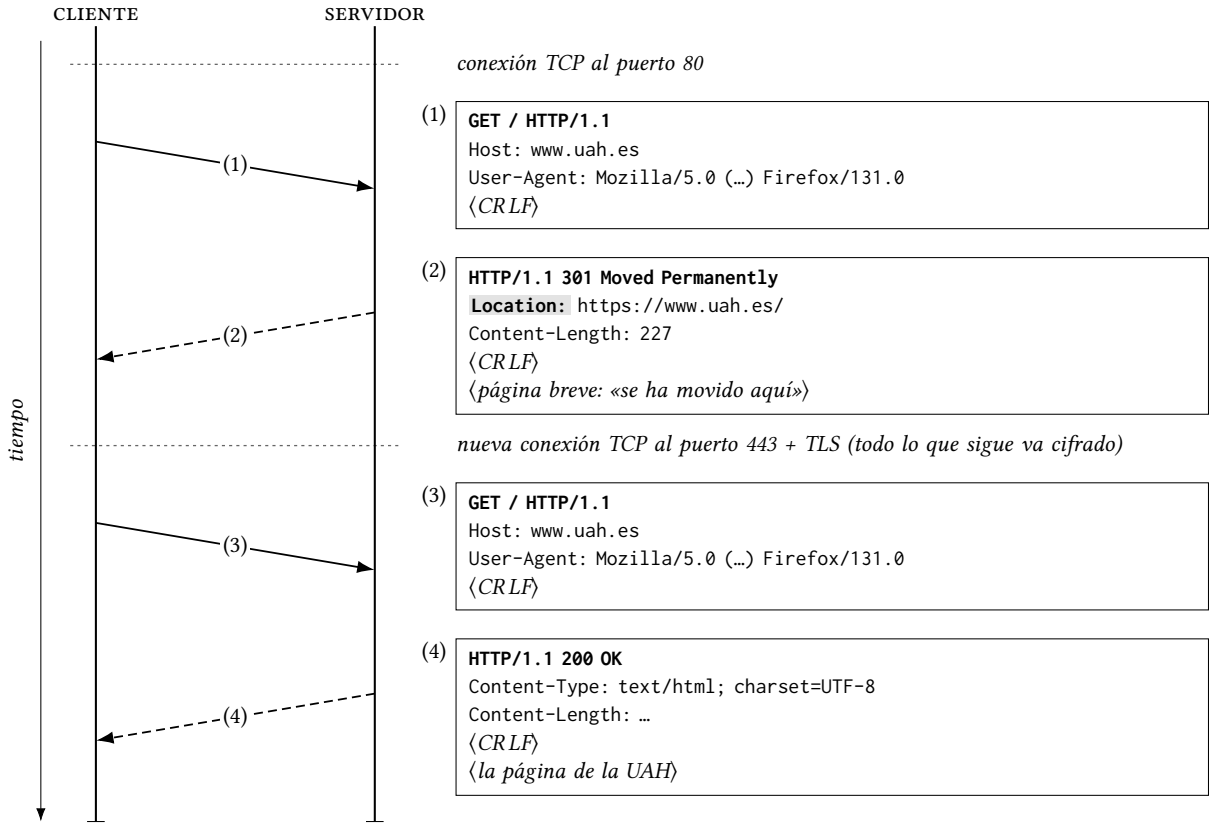


Figura 3. Redirección a HTTPS: 301 Moved Permanently.

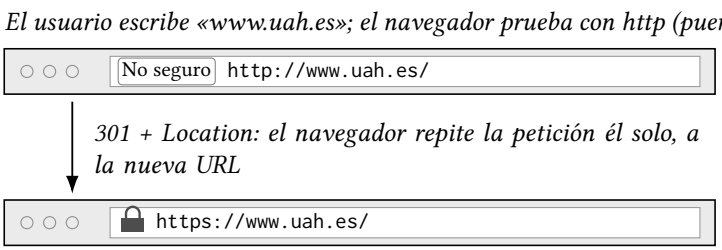


Figura 4. Redirección a HTTPS: 301 Moved Permanently: lo que pasa en el navegador.

OBSERVACIONES

(a) 3xx = hay que hacer algo más: la nueva URL va en Location.

(b) El navegador repite la petición él solo; el usuario no ve el 301.

(c) Datos reales: curl -v http://www.uah.es/ (15-oct-2026).

WEB DE PRÁCTICAS (`ar1.threatinteluah.com`):
captura: «Leer tráfico con proxy y varios servidores» (`CAP_HTTP_PROXY`)

4. Objeto inexistente: 404 Not Found

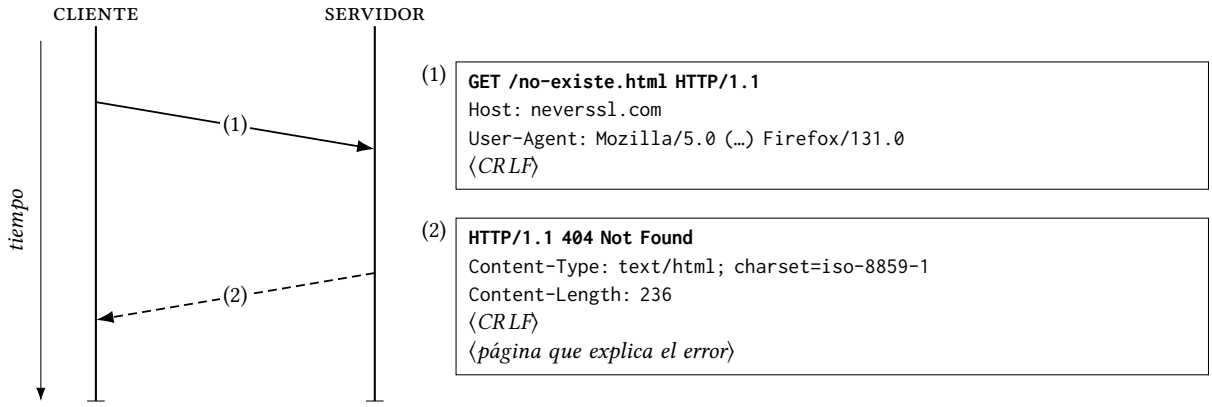


Figura 5. Objeto inexistente: 404 Not Found.

OBSERVACIONES

(a) 4xx = el error es del cliente (ha pedido algo que no existe).

(b) Una respuesta de error también lleva cuerpo, con su Content-Type y Content-Length.

(c) Si el fallo es del servidor: 5xx (500, 503).

5. Envío de un formulario: POST

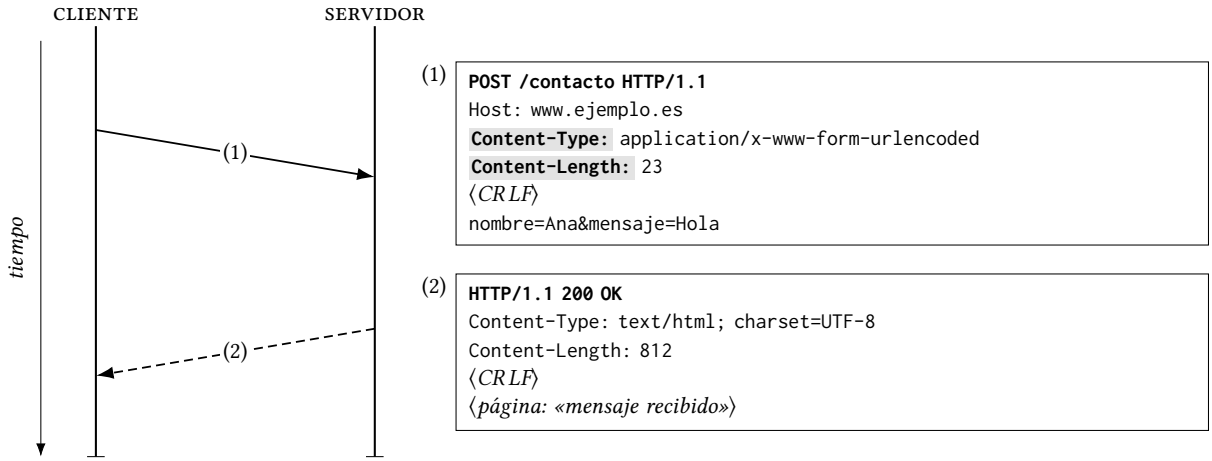


Figura 6. Envío de un formulario: POST.



Al pulsar «Enviar», el cuerpo de la petición (1) lleva:
`nombre=Ana&mensaje=Hola`

Figura 7. Envío de un formulario: POST: lo que pasa en el navegador.

OBSERVACIONES

(a) Los datos van en el CUERPO de la petición: por eso la petición lleva Content-Type y Content-Length.

(b) Con GET irían en la URL: GET /contacto?nombre=Ana&mensaje=Hola

(c) Content-Length: 23 = longitud de «nombre=Ana&mensaje=Hola».

6. Solo las cabeceras: HEAD

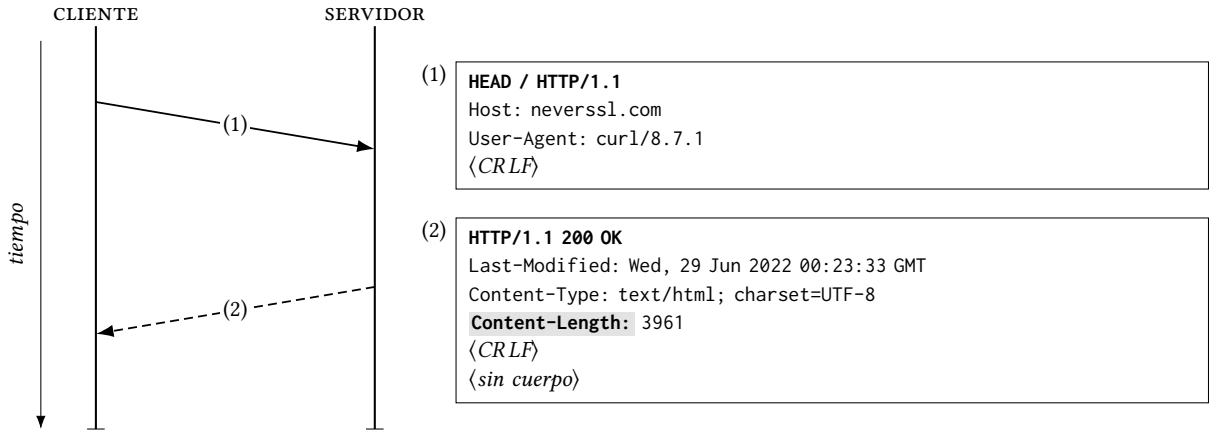


Figura 8. Solo las cabeceras: HEAD.

OBSERVACIONES

(a) Mismas cabeceras que con GET, pero sin cuerpo.

(b) Content-Length anuncia lo que ocuparía el cuerpo con GET.

(c) Sirve para comprobar si algo existe o ha cambiado. Pruébalo: curl -I http://neverssl.com/

7. GET condicional: 304 Not Modified

Este caso se estudia en la sesión GG23.

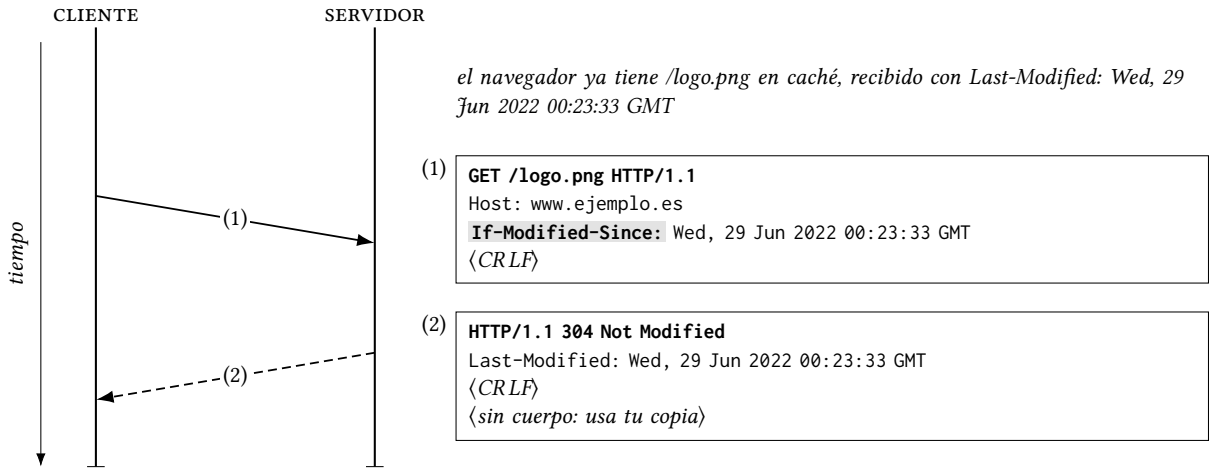


FIGURA 9. GET condicional: 304 Not Modified.

Caché del navegador

OBJETO	LAST-MODIFIED
www.ejemplo.es/logo.png	Wed, 29 Jun 2022 00:23:33 GMT

La petición (1) copia esa fecha en If-Modified-Since:
`If-Modified-Since: Wed, 29 Jun 2022 00:23:33 GMT`
304, sin cuerpo: el navegador muestra la copia que tiene en la caché.

FIGURA 10. GET condicional: 304 Not Modified: lo que pasa en el navegador.

OBSERVACIONES

(a) El cliente pregunta «¿ha cambiado desde esta fecha?».

(b) No ha cambiado ⟶ 304 sin cuerpo: se ahorra el objeto entero.

(c) Si hubiera cambiado ⟶ 200 OK con el objeto nuevo y un Last-Modified nuevo.

WEB DE PRÁCTICAS (`ar1.threatinteluah.com`):
captura: «Leer visitas repetidas con caché» (`CAP_HTTP_VISITA`)
secuencia: «Dibujar visitas con la caché del navegador» (`SEC_HTTP_CACHE`)

VISUALIZACIÓN INTERACTIVA: *Caché del navegador: ¿caché o servidor?*
`https://edelahozuah.github.io/computer-networks-visualizations/02-application-layer/http/http-cache.es.html`

8. Cookies: estado sobre un protocolo sin estado

Este caso se estudia en la sesión GG23.

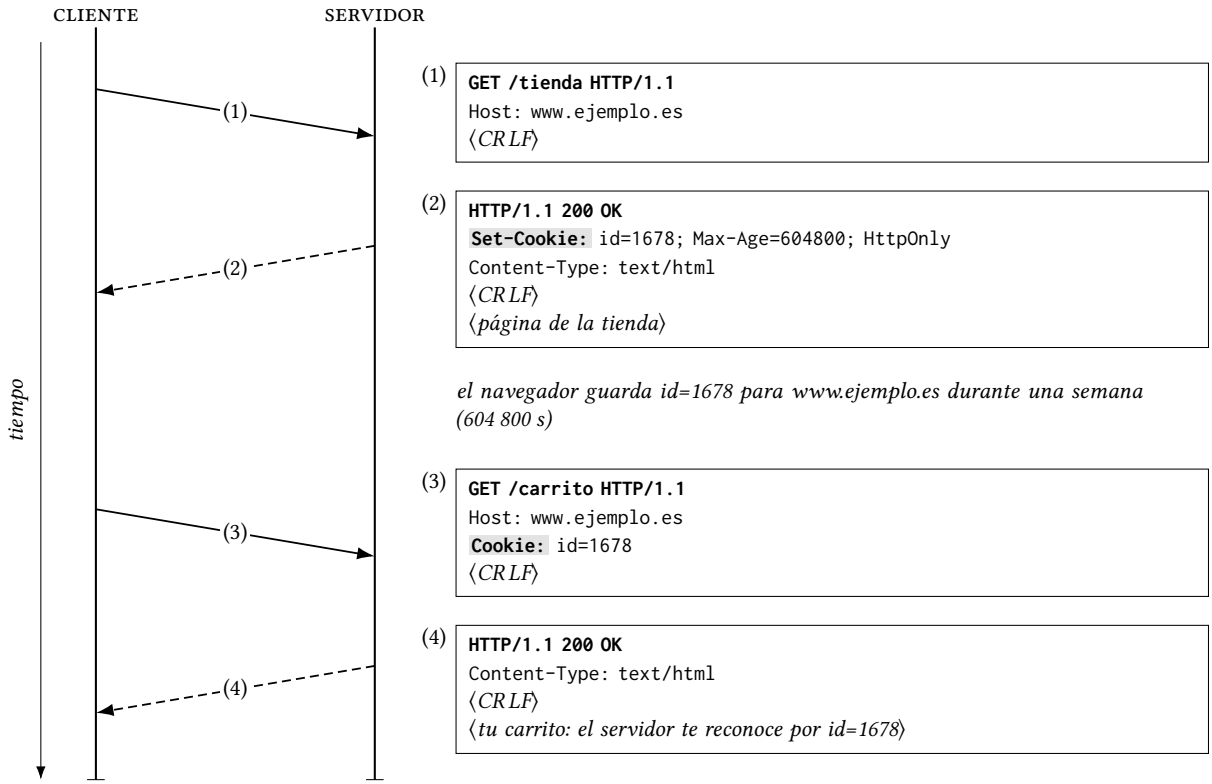


Figura 11. Cookies: estado sobre un protocolo sin estado.

Cookies guardadas en el navegador

DOMINIO	NOMBRE	VALOR	CADUCA
www.ejemplo.es	id	1678	dentro de 7 días

En cada petición siguiente a www.ejemplo.es, el navegador añade:
`Cookie: id=1678`

Figura 12. Cookies: estado sobre un protocolo sin estado: lo que pasa en el navegador.

OBSERVACIONES

(a) Set-Cookie va en una RESPUESTA (una vez); Cookie va en TODAS las peticiones siguientes a ese sitio.

(b) El servidor guarda qué significa id=1678 (en su base de datos); el navegador solo lo devuelve.

(c) Max-Age fija cuánto dura; sin él, la cookie se borra al cerrar el navegador.

WEB DE PRÁCTICAS (`ar1.threatinteluah.com`):
captura: «Leer visitas repetidas con caché» (`CAP_HTTP_VISITA`)
secuencia: «Dibujar visitas con cookies y autorización» (`SEC_HTTP_ESTADO`)

VISUALIZACIÓN INTERACTIVA: *Cookies: estado por encima de HTTP*
`https://edelahozuah.github.io/computer-networks-visualizations/02-application-layer/http/http-cookies.es.html`

9. Autorización básica: 401 Unauthorized

Este caso se estudia en la sesión GG23.

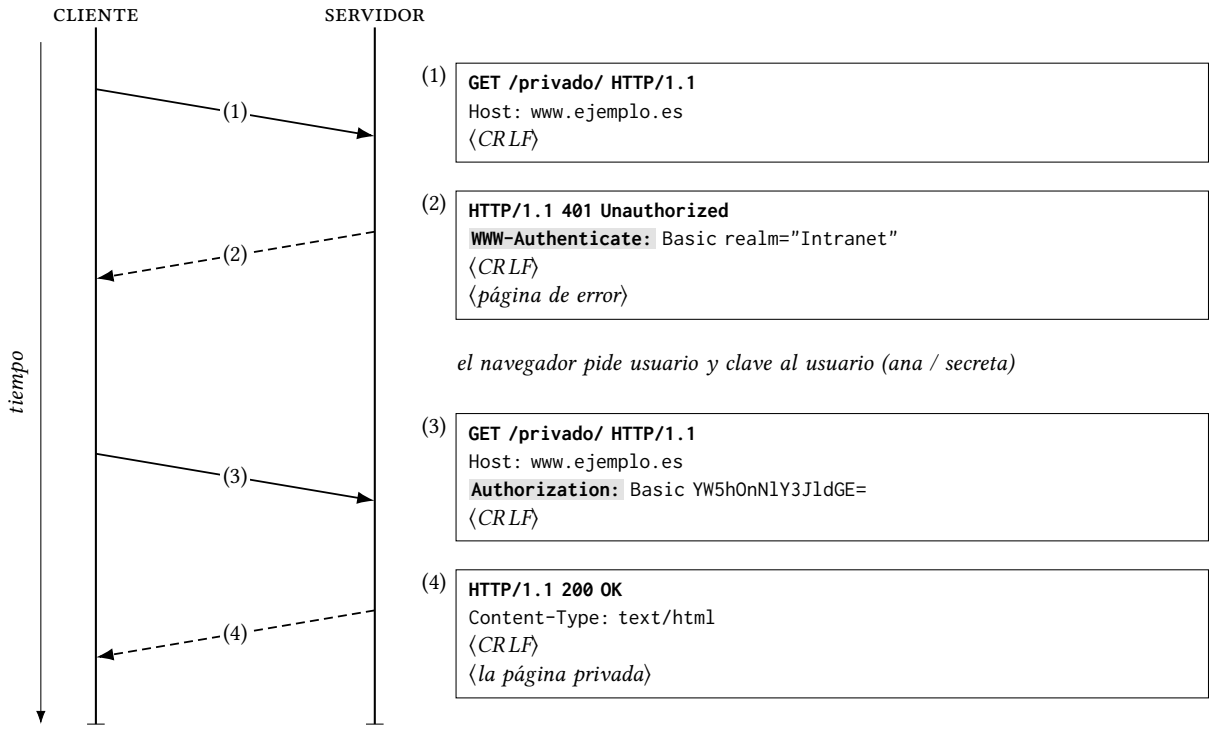


Figura 13. Autorización básica: 401 Unauthorized.



Al pulsar «Aceptar», la petición (3) lleva «ana:secreta» en Base64:
`Authorization: Basic YW5hOnNlY3JldGE=`

Figura 14. Autorización básica: 401 Unauthorized: lo que pasa en el navegador.

OBSERVACIONES

(a) YW5hOnNlY3JldGE= es «ana:secreta» en Base64: codificado, NO cifrado (se lee en cualquier captura).

(b) El navegador repite Authorization en cada petición siguiente al mismo sitio.

(c) Por eso la autorización básica solo es aceptable sobre HTTPS.

WEB DE PRÁCTICAS (`ar1.threatinteluah.com`):
captura: «Leer una petición y respuesta HTTP» (`CAP_HTTP_BASICA`)
secuencia: «Dibujar visitas con cookies y autorización» (`SEC_HTTP_ESTADO`)

10. Inicio de sesión por formulario: POST + cookie de sesión

Este caso se estudia en la sesión GG23 y en la práctica GP23 (parte 10).

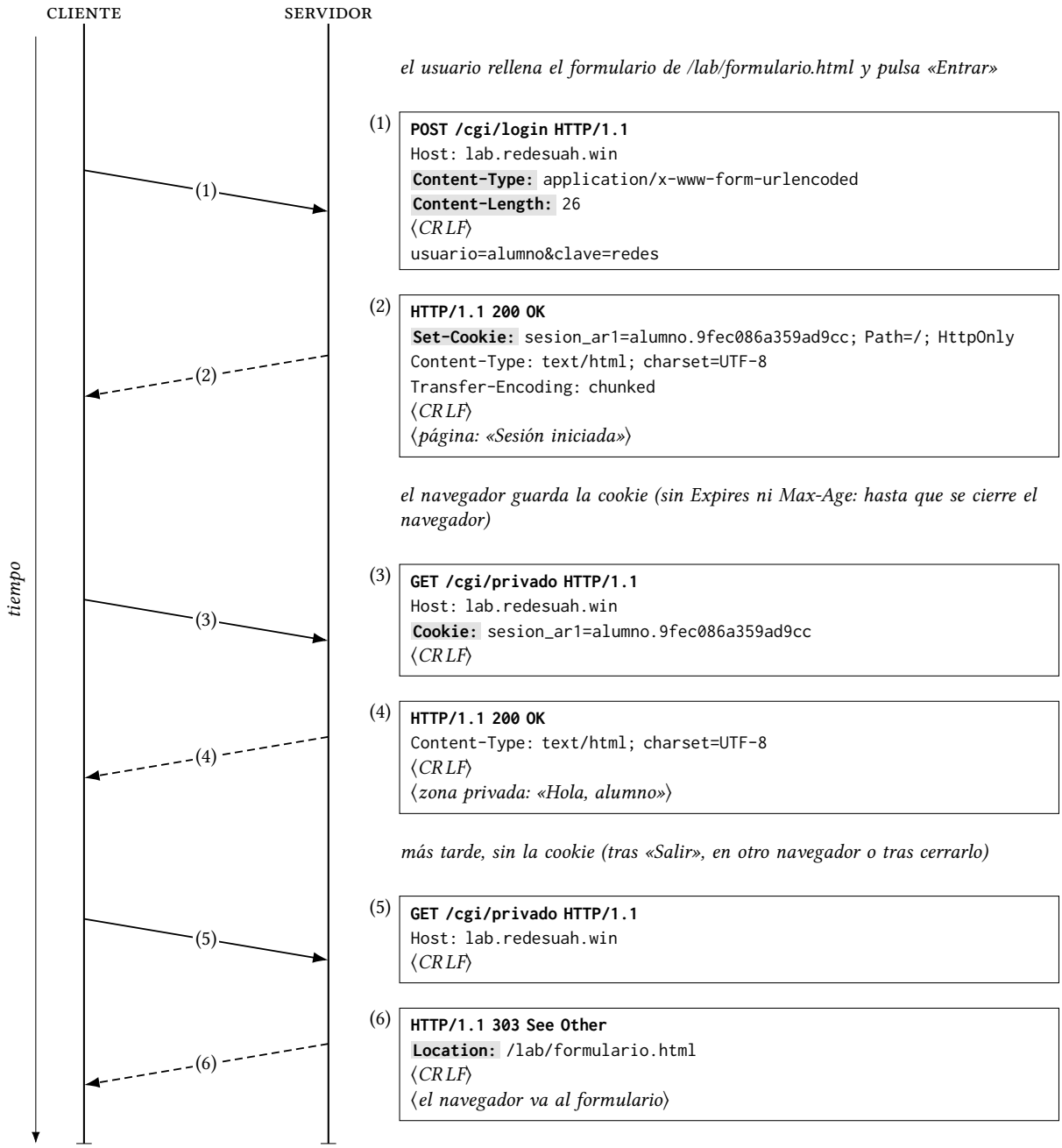
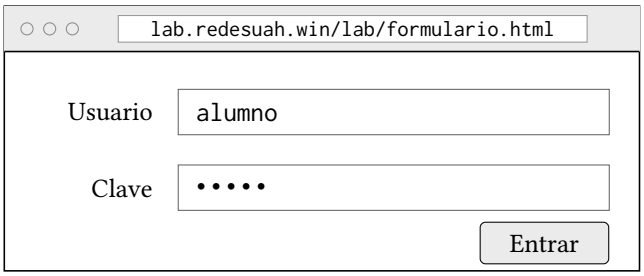


Figura 15. Inicio de sesión por formulario: POST + cookie de sesión.



Al pulsar «Entrar», el cuerpo de la petición (1) lleva:
`usuario=alumno&clave=redes`

Cookies guardadas en el navegador

DOMINIO	NOMBRE	VALOR	CADUCA
`lab.redesuah.win`	`sesion_ar1`	`alumno.9fec086a359ad9cc`	`al cerrar el navegador`

En cada petición siguiente a lab.redesuah.win, el navegador añade:
`Cookie: sesion_ar1=alumno.9fec086a359ad9cc`

FIGURA 16. Inicio de sesión por formulario: POST + cookie de sesión: lo que pasa en el navegador.

OBSERVACIONES

(a) La contraseña viaja UNA vez, en el cuerpo del POST (en claro: solo HTTPS la protege); después solo viaja la cookie.

(b) Con autorización básica (esquema 9), en cambio, Authorization viaja en cada petición.

(c) Sin cookie no hay 401: la sesión la gestiona la aplicación, que redirige al formulario (303 + Location).

(d) Datos reales de la captura de referencia de GP23 (lab.redesuah.win).

WEB DE PRÁCTICAS (`ar1.threatinteluah.com`):
secuencia: «Dibujar visitas con cookies y autorización» (`SEC_HTTP_ESTADO`)

VISUALIZACIÓN INTERACTIVA: *Cookies: estado por encima de HTTP*
`https://edelahozuah.github.io/computer-networks-visualizations/02-application-layer/http/http-cookies.es.html`